

Picomatch	Tests passing		
1	2	3	4
8 Table of Contents Click to expand - Install - Usage - API - picomatch - test - matchBase - isMatch - parse - scan - compileRe	7 - Accurate matching - Using wildcards (*) and (?), globstars (**) for nested directories, advanced globbing with extglobs, braces, and POSIX brackets, and support for escaping special characters with \ or quotes. - Well tested - Thousands of unit tests See the library comparison libraries.	6 Why picomatch? - Lightweight - No dependencies - Minimal - Tiny API surface. Main export is a function that takes a glob pattern and returns a matcher function. - Fast - Loads in about 2ms (that's several times faster than a single frame of a HD movie at 60fps) - Performant - Use the returned matcher function to speed up repeat matching (like when watching files)	5 Blazing fast and accurate glob matcher written in JavaScript. No dependencies and full support for standard and extended Bash glob features, including braces, extglobs, POSIX brackets, and regular expressions.
9 - Library Comparisons - Matching special characters as literals - Braces - Advanced globbing - Basic globbing - Globbing features - Options Examples - Scan Options - Picomatch options - Options - toRegex - makeRe	10 Install Install with npm: - License - Author - About - Philosophies - Benchmarks (TOC generated by verb using markdown-toc)	11 Usage The main export is a function that takes a glob pattern and an options object and returns a function for matching strings. const pm = require('picomatch'); const isMatch = pm('*.*.js');	12 console.log(isMatch('abcd')); //=> false console.log(isMatch('a.js')); //=> true console.log(isMatch('a.md')); //=> false console.log(isMatch('a/b.js')); //=> false
16 {Object}: Returns an object with matching info. Example const picomatch = require('picomatch'); // picomatch.test(input, regex[, options]);	15 - regex {RegExp} - input {String}: String to test. Params test the input string. Test input with the given regex. This is used by the main picomatch() function to Test console.log(isMatch('a.a')); //=> false console.log(isMatch('a.b')); //=> true	14 - globs {String Array}: One or more glob patterns. {Object}: Returns a matcher function. {Function=} {Function=} Example const picomatch = require('picomatch'); // picomatch(glob[, options]); const isMatch = picomatch('*.*(a)');	13 API Creates a matcher function from one or more glob patterns. The returned function takes a string to match as its first argument, and returns true if the string is a match. The returned matcher function also takes a boolean as the second argument that, when true, returns an object with additional information.

[illegible]

Option	Type	Default value	Description	capture	boolean	undefined	Backslash escape and treat stars as (**). Return matched support method
				contains	boolean	undefined	
						33	
Option	Type	Default value	Description	cwd	string	process.cwd()	Allows match the given directory
				debug	boolean	undefined	Current directory
				dot	boolean	false	Debug expression an error
						34	
Option	Type	Default value	Description	expandRange	function	undefined	Enable matching default ignored is explicit in the pattern Custom expand brace pattern such as {a,b}
						35	
						36	
Option	Type	Default value	Description	fastpaths	boolean	true	The fastpaths for a hash parsing process To speed same regex found. Throws
				failglob	boolean	false	same as bash glob found. no match
						37	
Option	Type	Default value	Description	format	function	undefined	Custom format will be the node regex. In the glob setting this behavior
				flags	string	undefined	Regex flags
						38	
Option	Type	Default value	Description	ignore	string	undefined	Ignore array\string pattern that should be excluded
				keepQuotes	boolean	false	One or more Posix path slashes removed. This is returned
				literalBrackets	boolean	undefined	When brackets are used in the pattern, also be generated. Retain result. matched
						40	
Option	Type	Default value	Description	matchBase	boolean	false	Match pattern escape only literal will be Alias for Limit the length string. thrown
				maxLength	boolean	65536	
						41	
Option	Type	Default value	Description	nobrace	boolean	false	Disable matching {a,b} would be literal Disable with re bracket
				nobacket	boolean	undefined	
						42	
Option	Type	Default value	Description	onIgnore	function	undefined	Function called when the pattern is ignored
				onResult	function	undefined	Function called when the pattern is matched
				posix	boolean	false	Posix style path separator
						47	
Option	Type	Default value	Description	onMatch	function	undefined	Function called when the pattern is matched
				nodupes	boolean	true	Disable matching of duplicate patterns
				nocase	boolean	false	Make no case-insensitive Equivalent regex flags that the override flags Deprecation This option remove
						43	
Option	Type	Default value	Description	noextglob	boolean	false	Disable matching of extended glob (like a,b) (b) extglob
				noextstar	boolean	false	Disable matching of extended star
				negate	boolean	false	Disable negating
				noquantifiers	boolean	false	Disable quantifiers
						45	
Option	Type	Default value	Description	capture	boolean	undefined	Backslash escape and treat stars as (**). Return matched support method
				contains	boolean	undefined	
						48	

isMatch('bar'); isMatch('baz'); options.onIgnore <pre>const onIgnore = ({ glob, regex, input, output }) => { console.log({ glob, regex, input, output }); };</pre> 65	const isMatch = picomatch('*', { onIgnore, ignore: 'f*' }); isMatch('foo'); isMatch('bar'); isMatch('baz'); options.onResult <pre>const onResult = ({ glob, regex, input, output }) => { console.log({ glob, regex, input, output }); };</pre> 66	}; const isMatch = picomatch('*', { onResult, ignore: 'f*' }); isMatch('foo'); isMatch('bar'); isMatch('baz'); 67	Globbering features - Basic globbing (Wildcard matching) - Advanced globbing (extglobs, posix brackets, brace matching) Basic globbing Character Description * Matches any character zero or more times, 68
Character Description Matches any path separators, including character excluding path separators one time. Does not match path separators or leading dots. Matches any characters inside the brackets. For example, [abc] would match the 72	Character Description bar, and foo/a**b/ foo/a*b/bar, and more than two consecutive stars in a glob path segment are regarded as a single star. Thus, foo/**/bar is equivalent to foo/*/* 71	Character Description more times, including path separators. Note that ** will only match path separators (/ , and \ on Windows) when they are the only characters in a path segment. Thus, foo**/bar is equivalent to foo/* 70	Character Description excluding path separators. Does not match path separators or hidden files or directories ("dotfiles"), unless explicitly enabled by setting the dot option to true. Matches any character zero or 69
Character Description Picomatch's matching features and expected results in unit tests are based on Matching behavior vs. Bash 73	Bash's unit tests and the Bash 4.3 specification, with the following exceptions: - Bash will match foo/bar/baz with *. - Picomatch only matches nested directories with **. - Bash greedily matches with negated extglobs. For example, Bash 4.3 says that ! (foo)* should match foo and foobar, since the trailing * backtracks to match the preceding pattern. This is very memory-inefficient, and IMHO, also incorrect. 74	Picomatch would return false for both foo and foobar. Advanced globbing - extglobs - POSIX brackets - Braces 75	Extglobs Pattern Description @(pattern) Match only one consecutive occurrence of pattern *(pattern) Match zero or more consecutive occurrences of pattern 76
POSIX brackets POSIX classes are disabled by default. Enable this feature by setting the posix option to true. Enable POSIX bracket support 80	// +(pattern) matches ONE or more of "pattern" console.log(pm.isMatch('a', 'a*(z)'); // true console.log(pm.isMatch('az', 'a*(z)'); // true console.log(pm.isMatch('aazz', 'a*(z)'); // true // supports multiple extglobs console.log(pm.isMatch('foo.bar', '!(foo).!(bar)'); // false 79	Examples <pre>const pm = require('picomatch'); // *(pattern) matches ZERO or more of "pattern" console.log(pm.isMatch('a', 'a*(z)'); // true console.log(pm.isMatch('az', 'a*(z)'); // true console.log(pm.isMatch('aazz', 'a*(z)'); // true // supports multiple extglobs console.log(pm.isMatch('foo.bar', '!(foo).!(bar)'); // false</pre> 78	Pattern Description +(pattern) Match one or more consecutive occurrences of pattern ?(pattern) Match zero or one consecutive occurrences of pattern i(pattern) Match anything but pattern 77

Please read the [contributing guide](#) for advice on opening issues, pull requests, and coding standards.

Running Tests

Running and reviewing unit tests is a great way to get familiarized with a library and its API. You can install dependencies and run tests with the following command:

```
npm install && npm test
```

Building docs

(This project's readme.md is generated by [verb](#), please don't edit the readme directly. Any changes to the readme must be made in the [.verb.md](#) readme template.)

To generate the readme, run the following command:

```
npm install -g verbose/verb#dev verb-  
generate-readme && verb
```

Author

Jon Schlinkert

- [GitHub Profile](#)
- [Twitter Profile](#)
- [LinkedIn Profile](#)

License

Copyright © 2017-present, [Jon Schlinkert](#).
Released under the [MIT License](#).

Please read the contributing guide for advice on opening issues, pull requests, and coding standards. Running Tests Running and reviewing unit tests is a great way to get familiarized with a library and its API. You can install dependencies and run tests with the following command: <pre>npm install && npm test</pre> Building docs	<i>(This project's <code>readme.md</code> is generated by verb, please don't edit the <code>readme</code> directly. Any changes to the <code>readme</code> must be made in the .verb.md <code>readme</code> template.)</i> To generate the <code>readme</code>, run the following command: <pre>npm install -g verbose/verb#dev verb-generate-readme && verb</pre>	Author Jon Schlinkert • GitHub Profile • Twitter Profile • LinkedIn Profile License Copyright © 2017-present, Jon Schlinkert. Released under the MIT License.	
97	98	99	100
†01	£01	201	101
105	106	107	108
211	111	011	601